

---

# Rapport de Déploiement

Plateforme GANDAL — Frontend Web

*Mise en production sur le data center local*

---

|               |                                                                                           |
|---------------|-------------------------------------------------------------------------------------------|
| Application   | Frontend Next.js 16 (App Router)                                                          |
| Serveur cible | root@10.50.30.102 (Debian 12)                                                             |
| URL publique  | <a href="http://gandal-app.10.50.30.102.nip.io">http://gandal-app.10.50.30.102.nip.io</a> |
| Méthode       | Conteneurisation Docker + reverse proxy nginx                                             |
| Date          | 11 juin 2026                                                                              |
| Statut        | Déployé et vérifié                                                                        |

École Nationale Supérieure Polytechnique de Yaoundé (ENSPY)

Département de Génie Informatique — Promotion GI27

Université de Yaoundé I

## Table des matières

|          |                                              |          |
|----------|----------------------------------------------|----------|
| <b>1</b> | <b>Contexte et objectif</b>                  | <b>2</b> |
| 1.1      | Périmètre . . . . .                          | 2        |
| <b>2</b> | <b>Environnement cible</b>                   | <b>2</b> |
| 2.1      | Justification du choix Docker . . . . .      | 2        |
| <b>3</b> | <b>Architecture de déploiement</b>           | <b>2</b> |
| <b>4</b> | <b>Artefacts de déploiement</b>              | <b>3</b> |
| 4.1      | Dockerfile (multi-stage) . . . . .           | 3        |
| 4.2      | Orchestration (docker-compose.yml) . . . . . | 3        |
| <b>5</b> | <b>Procédure de déploiement</b>              | <b>3</b> |
| 5.1      | Transfert des sources . . . . .              | 3        |
| 5.2      | Construction et démarrage . . . . .          | 4        |
| 5.3      | Routage nginx . . . . .                      | 4        |
| 5.4      | Configuration CORS . . . . .                 | 4        |
| <b>6</b> | <b>Incident rencontré et résolution</b>      | <b>4</b> |
| 6.1      | Symptôme . . . . .                           | 4        |
| 6.2      | Cause racine . . . . .                       | 4        |
| 6.3      | Résolution . . . . .                         | 4        |
| 6.4      | Recommandation . . . . .                     | 5        |
| <b>7</b> | <b>Vérification fonctionnelle</b>            | <b>5</b> |
| <b>8</b> | <b>Exploitation et maintenance</b>           | <b>5</b> |
| 8.1      | Commandes usuelles . . . . .                 | 5        |
| 8.2      | Procédure de rollback . . . . .              | 5        |
| <b>9</b> | <b>Conclusion</b>                            | <b>6</b> |

## 1 Contexte et objectif

GANDAL est une plateforme académique de gestion centralisée de projets étudiants, adossée à un data center local du Département de Génie Informatique. Le présent document décrit la mise en production de la **couche frontend** (interface web) sur le serveur d'hébergement 10.50.30.102, aux côtés de l'API backend déjà en service.

### 1.1 Périmètre

- Construction d'une image Docker de l'application Next.js.
- Démarrage du conteneur sur l'infrastructure existante.
- Exposition via le reverse proxy nginx mutualisé.
- Configuration du partage de ressources cross-origin (CORS) côté API.
- Vérification fonctionnelle de bout en bout.

## 2 Environnement cible

| Élément               | Valeur                                                           |
|-----------------------|------------------------------------------------------------------|
| Hôte                  | omega-test-3001, Debian GNU/Linux 12, x86_64                     |
| Moteur de conteneurs  | Docker 29.5.3 + Docker Compose v5.1.4                            |
| Node.js sur l'hôte    | <i>Absent</i> (d'où le choix de la conteneurisation)             |
| Reverse proxy         | Conteneur <b>iwm-gateway</b> (nginx 1.29.8-alpine), port 80      |
| Backend existant      | <b>gandal-api</b> (port 8000) + <b>gandal-db</b> (PostgreSQL 17) |
| Réseau Docker partagé | <b>iwm-vm-network</b> (résolution par nom de service)            |

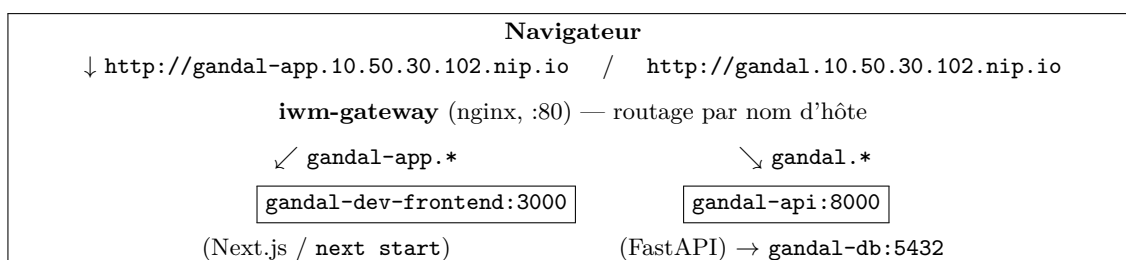
TABLE 1 – Caractéristiques du serveur de production.

### 2.1 Justification du choix Docker

L'ensemble de l'infrastructure du serveur est conteneurisé : le reverse proxy nginx route le trafic vers des *noms de service Docker* sur le réseau interne. Le serveur ne disposant pas de runtime Node.js, et un export statique étant impossible (l'application utilise un *middleware* d'authentification exécuté côté serveur), la conteneurisation est la solution cohérente et la moins intrusive, alignée sur le déploiement du backend.

## 3 Architecture de déploiement

Le frontend est exposé sur un **hôte virtuel dédié** distinct de l'API, le routage étant assuré par la directive `map $host $svc` du reverse proxy.



- `gandal-app.10.50.30.102.nip.io` → conteneur frontend (port 3000).
- `gandal.10.50.30.102.nip.io` → API backend (port 8000), inchangé.
- Les appels API du navigateur étant cross-origin, le backend autorise explicitement l'origine du frontend (cf. §5.4).

## 4 Artefacts de déploiement

### 4.1 Dockerfile (multi-stage)

Image construite en deux étapes : compilation puis exécution avec l'utilisateur non privilégié `node`. L'approche s'appuie sur les scripts du projet (`next build` / `next start`), robuste vis-à-vis des spécificités de cette distribution de Next.js.

```
FROM node:20-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY . .
ENV NEXT_TELEMETRY_DISABLED=1
RUN npm run build
RUN npm prune --omit=dev

FROM node:20-alpine AS runner
WORKDIR /app
ENV NODE_ENV=production PORT=3000 HOSTNAME=0.0.0.0
COPY --from=build --chown=node:node /app/package.json ./package.json
COPY --from=build --chown=node:node /app/node_modules ./node_modules
COPY --from=build --chown=node:node /app/.next ./next
COPY --from=build --chown=node:node /app/public ./public
COPY --from=build --chown=node:node /app/next.config.ts ./next.config.ts
USER node
EXPOSE 3000
CMD ["npm", "run", "start"]
```

### 4.2 Orchestration (docker-compose.yml)

```
name: gandal-frontend
services:
  frontend:
    build: { context: ., dockerfile: Dockerfile }
    image: gandal-dev-frontend:latest
    container_name: gandal-dev-frontend
    restart: unless-stopped
    environment: { NODE_ENV: production, PORT: "3000", HOSTNAME: 0.0.0.0 }
    expose: ["3000"]
    networks: [gateway-net]
networks:
  gateway-net:
    external: true
    name: iwm-vm-network
```

## 5 Procédure de déploiement

### 5.1 Transfert des sources

Les sources (≈11 Mo, assets statiques inclus) sont synchronisées vers `/root/gandal-dev-frontend` :

```
rsync -az --delete --exclude=node_modules --exclude=.next --exclude=.git \  
./ root@10.50.30.102:/root/gandal-dev-frontend/
```

## 5.2 Construction et démarrage

La construction est réalisée directement sur le serveur (build isolé dans le conteneur, indépendant de l'environnement hôte) :

```
cd /root/gandal-dev-frontend && docker compose up -d --build
```

Résultat : image `gandal-dev-frontend:latest` (≈911 Mo), conteneur `gandal-dev-frontend` en état `running` sur le réseau `iwm-vm-network`.

## 5.3 Routage nginx

Ajout d'une entrée dans `/root/iwm/ops/nginx/gateway.conf` :

```
map $host $svc {  
    default "";  
    "~^gandal-app\.\" "gandal-dev-frontend:3000"; # <-- ajout  
    "~^gandal(-api)?\.\" "gandal-api:8000";  
    ...  
}
```

## 5.4 Configuration CORS

L'origine du frontend est ajoutée à `CORS_ORIGINS` dans `/gandal-dev/.env`, puis le service API est recréé pour prise en compte :

```
CORS_ORIGINS=...,http://gandal-app.10.50.30.102.nip.io  
cd /gandal-dev && docker compose -f docker-compose.prod.yml up -d api
```

# 6 Incident rencontré et résolution

## 6.1 Symptôme

Après ajout de la règle de routage, l'URL du frontend renvoyait HTTP 404 - `Unknown host`, alors que la configuration sur disque était correcte et que la résolution DNS interne du conteneur fonctionnait.

## 6.2 Cause racine

Le conteneur `iwm-gateway` monte un *fichier unique* (`gateway.conf`) en *bind mount*. Une modification via `sed -i` remplace l'*inode* du fichier; le conteneur reste alors accroché à l'ancien inode. Les rechargements `nginx -s reload` relisaient donc une configuration figée, sans la nouvelle règle.

## 6.3 Résolution

Validation de la configuration dans un conteneur `nginx` jetable, puis redémarrage du `gateway` pour re-lie le fichier :

```
docker run --rm -v /root/iwm/ops/nginx/gateway.conf:\
/etc/nginx/conf.d/default.conf:ro nginx:1.29.8-alpine nginx -t # OK
docker restart iwm-gateway
```

## 6.4 Recommandation

Pour toute modification ultérieure du gateway, privilégier une réécriture *in-place* préservant l'inode, ou prévoir le redémarrage du conteneur. À terme, monter un *répertoire* plutôt qu'un fichier unique élimine ce risque.

## 7 Vérification fonctionnelle

| Contrôle                       | Attendu                | Résultat                        |
|--------------------------------|------------------------|---------------------------------|
| Page d'accueil via gateway     | HTTP 200               | <b>200</b> (HTML, titre GANDAL) |
| Asset statique /logo.png       | HTTP 200               | <b>200</b> (112 Ko)             |
| API gandal.*/health            | HTTP 200               | <b>200</b> (backend intact)     |
| Préflight CORS depuis le front | En-tête autorisé       | <b>allow-origin présent</b>     |
| Persistance conteneur          | restart=unless-stopped | <b>Actif</b>                    |

TABLE 2 – Synthèse des tests post-déploiement.

En-tête CORS retourné par l'API pour l'origine du frontend :

```
access-control-allow-origin: http://gandal-app.10.50.30.102.nip.io
access-control-allow-credentials: true
access-control-allow-methods: DELETE, GET, HEAD, OPTIONS, PATCH, POST, PUT
```

## 8 Exploitation et maintenance

### 8.1 Commandes usuelles

```
# Etat / logs
docker ps --filter name=gandal-dev-frontend
docker logs -f gandal-dev-frontend

# Mise à jour (après rsync des nouvelles sources)
cd /root/gandal-dev-frontend && docker compose up -d --build

# Redémarrage
docker restart gandal-dev-frontend
```

### 8.2 Procédure de rollback

1. Restaurer la sauvegarde du gateway : `gateway.conf.bak.*` puis `docker restart iwm-gateway`.
2. Restaurer `/gandal-dev/.env.bak.*` et recréer l'API.
3. Arrêter le frontend : `docker compose down` dans `/root/gandal-dev-frontend`.

## 9 Conclusion

Le frontend GANDAL est déployé en production sur 10.50.30.102 et accessible à l'adresse <http://gandal-app.10.50.30.102.nip.io>. L'intégration au reverse proxy mutualisé et la configuration CORS permettent une communication complète avec l'API existante, sans régression sur les autres services hébergés. L'ensemble est géré par Docker Compose, avec redémarrage automatique et procédure de rollback documentée.

### Points d'amélioration suggérés

- Mise en place de TLS (HTTPS) sur le gateway.
- Intégration continue : build et publication de l'image vers un registre (ex. GHCR) pour réduire la charge de build sur le serveur.
- Surveillance de la mémoire (RAM limitée, absence de swap sur l'hôte).

*Document généré dans le cadre du déploiement du 11 juin 2026.*